Barliesque.SceneManagement

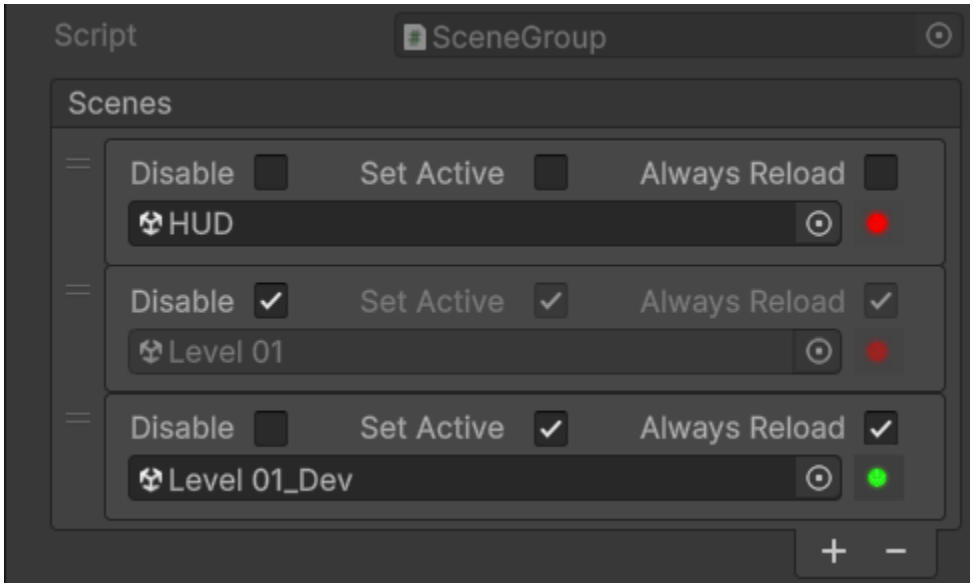
This package provides a convenient API and management system to load and unload your application's scenes.

Start Scene

The first scene in Unity's scene list is always loaded first, so this is where the `SceneLoader` component should be placed, to take over further loading of scenes. The `SceneLoader` will ***never*** unload this scene, so it's an ideal place for many other application-wide management components, such as analytics, audio and localization management ...*but no actual content*. As soon as this scene is loaded, we can get on with loading the initial content scene. First, we need to create some `SceneGroups`.

Creating `SceneGroups`

Whether you are loading individual or multiple scenes at once, you must create a SceneGroup object to specify them. In the Project panel, right click and select **Create » Barliesque » Scene Group**.



This example SceneGroup includes multiple scenes to be loaded together. Note that the traffic lights to the right of each selected scene are alerting us to an issue; the HUD scene hasn't been added to Unity's build settings Scene List, which means it won't be included in the build and attempting to load it will throw an error. The Level 01 scene is missing as well, but it's

been disabled and won't be loaded. The **Disable** option allows you to stop a scene from being loaded, without having to remove it from the `SceneGroup`.

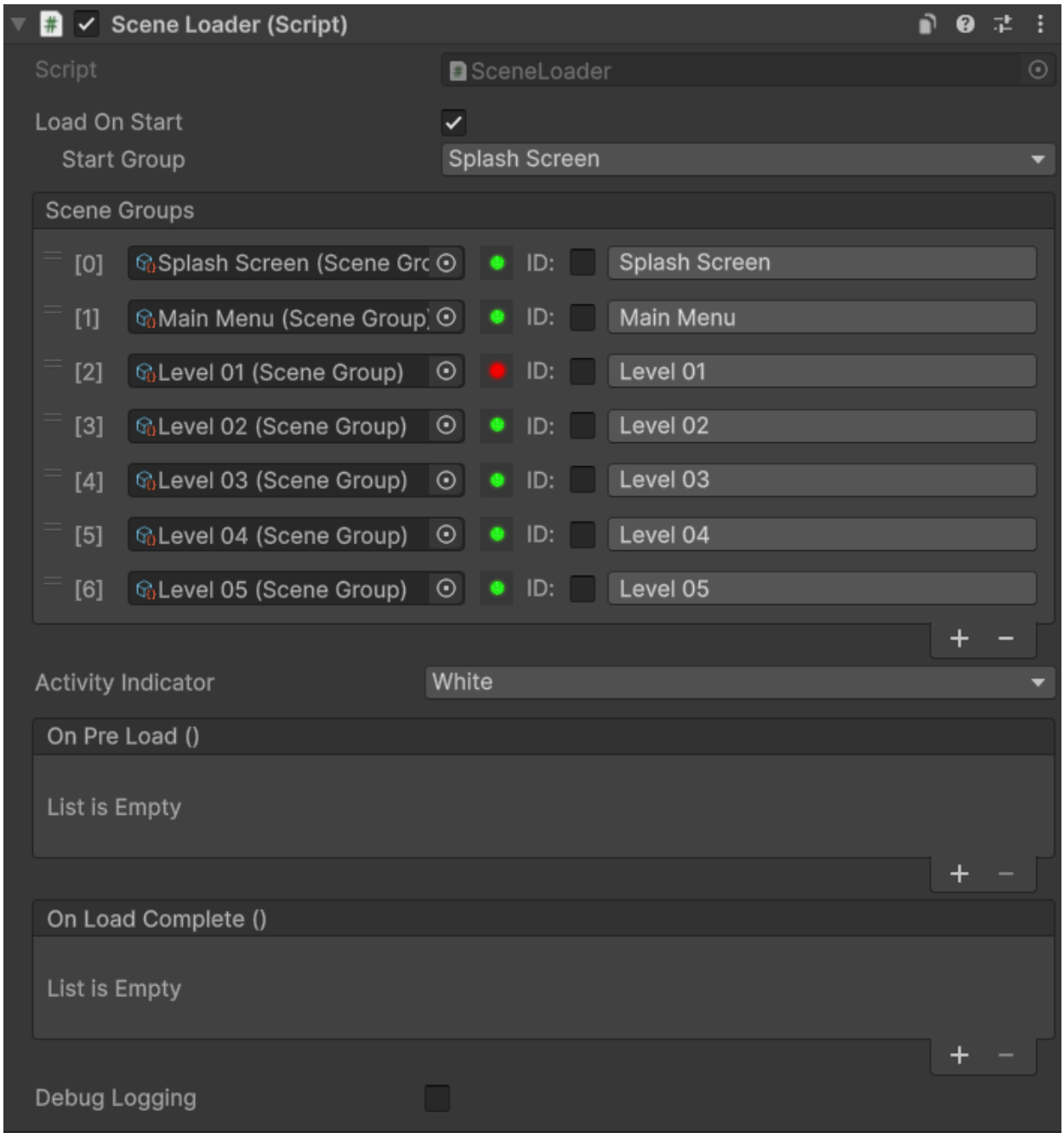
One scene in any `SceneGroup` should always be marked as the **Active Scene** — this is the primary scene Unity will designate to receive new `GameObjects` instantiated at runtime (via `Instantiate`), and it acts as the source for lighting settings. Since Level 01 isn't enabled, only the dev version of the scene will be loaded and set as the active scene.

In some situations, when loading between different `SceneGroups`, there may be one scene — for instance, a heads-up display — that's used in both the outgoing and incoming `SceneGroups`. To make for a more seamless experience, you may prefer that scene to not be reloaded. If, on the other hand, it should always be reloaded, you can enable the **Always Reload** option.

Unity needs to process light probes after loading scenes. Unity creates a 3D mesh (tetrahedron network) from light probe positions, to determine how dynamic objects are lit. When loading scenes additively, as this library does, Unity does not automatically update the light probe tetrahedral tessellation. If `LightProbes.Tetrahedralize()` is not called, dynamic objects may use old or incorrect light probe data from previous scenes, causing improper lighting on them. Enabling the **Tetrahedralize Probes** option in the `SceneGroup` takes care of this as part of the loading process. If for whatever reason, your scene does not require tetrahedralization—computationally expensive—leave this option disabled.

Setting up the `SceneLoader`

Start by dragging your `SceneGroup` objects into the `SceneLoader`'s list.



As you can see, the issue we saw in the Level 01 `SceneGroup` shows up in the `SceneLoader` too. In case a `SceneGroup` references a scene that isn't in the Unity build settings Scene List, that alert is surfaced here as well as in the affected `SceneGroup` object to help you find and fix the problem.

The first parameter of the `SceneLoader` component, **Load on Start** allows you to automatically load in a scene group that you specify from the list.

SceneLoader Scripting API

The `SceneLoader` is a singleton class with a straight-forward static API--a direct link to the instance is not necessary. There are three ways that `SceneGroups` can be referenced to be loaded—for instance, using the **string ID** set in the `SceneLoader`:

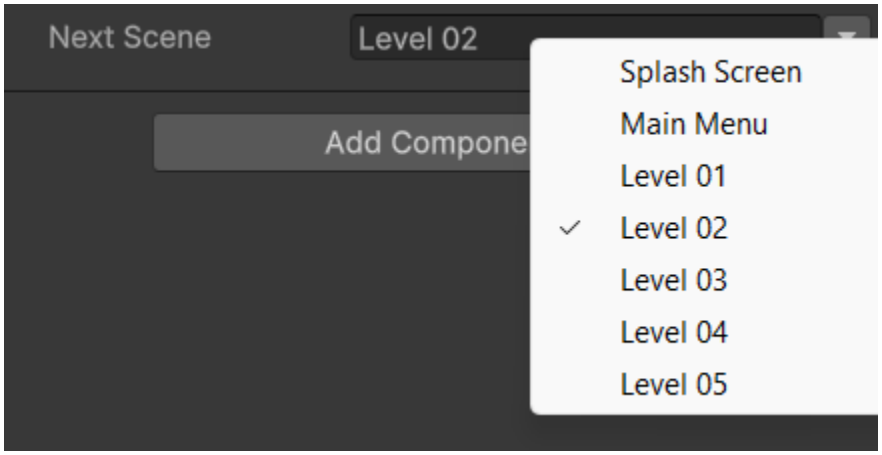
```
private void LoadMainMenu()
{
   SceneLoader.Load("Main Menu");
}
```

Any scenes that had already been loaded (and that are not part of the "MainMenu" group) will first be unloaded, and then the scenes in the "MainMenu" group will be loaded. Again, the start scene, containing the `SceneLoader` itself, is never unloaded.

The `[SceneGroupID]` attribute adds a handy dropdown in the inspector, to make selection easy and accurate.

```
[SerializeField, SceneGroupID] private string _nextScene;

private void LoadNextScene()
{
   SceneLoader.Load(_nextScene);
}
```



Alternatively, you can use the **index** in the `SceneLoader`. This is not generally recommended, since its value could easily change if that list were to be modified at a later date—which of course could cause the wrong `SceneGroup` to get loaded. However, suppose you have a game with 100 levels—using an index could work very well.

```
private void LoadLevel(int level)
{
   var first = SceneLoader.GetSceneIndex("Level 01");
   var index = first + level - 1;
   SceneLoader.Load(index);
}
```

You can also use a **reference** to the `SceneGroup` itself—which is the safest way of all, though not as adaptable as using the string ID.

```
[SerializeField] private SceneGroup _nextScene;

private void LoadNextScene()
{
   SceneLoader.Load(_nextScene);
}
```

SceneLoader Events

An event is invoked before and after loading, to enable the control of full-screen transitions for seamless scene changes, for example. Those events are accessible both from the inspector and via the static API of the class.

```
public class SceneTransition : MonoBehaviour
{
   private void Start()
   {
      SceneLoader.OnPreLoad.AddListener(CoverScreenAnimation);
      SceneLoader.OnLoadComplete.AddListener(RevealScreenAnimation);
   }

   private void OnDestroy()
   {
      SceneLoader.OnPreLoad.RemoveListener(CoverScreenAnimation);
      SceneLoader.OnLoadComplete.RemoveListener(RevealScreenAnimation);
   }
```